



INSTITUTO POLITECNICO NACIONAL

Escuela Superior de Computo

Arquitectura de las Computadoras

Monociclo en Verilog y Rars

Profesor: Miguel Angel Aleman Arce

Alumno: Gutiérrez Espinosa Jordan Gabriel

Grupo: 5CV1

Ciclo: 2022-2023(1)

Introducción

Un microprocesador simple tipo Monociclo de 4 bits, que cuente con 9 instrucciones, tiene 2 puertos (uno de entrada y otro de salida) de 4 bits cada uno. El Microprocesador consta de 12módulos: control, control pc, PC, memprog, ALU, archivo,

registros, sumador, sumador1, mux1, mux2, mux3, buftri1. Las instrucciones son 9, las cuales están divididas en 4 grupos, por otro lado, se tienen 3 formatos diferentes de instrucción, pero todas con una longitud de palabra de 10 bits, además cada una lleva su propio “opcode” [código de operación].

se caracteriza porque la ejecución de cada instrucción tiene una duración de un ciclo de reloj. Como el ciclo de reloj debe estar adaptado a las necesidades de la ejecución de todas las instrucciones y necesita ajustar su duración a la de aquella instrucción que sea más larga. Pero el tiempo necesario para la ejecución puede variar sensiblemente de una instrucción a otra. Así una jump o salto incondicional necesita mucho menos tiempo que, por ejemplo, una carga. El concentrar la totalidad de la ejecución en un ciclo de reloj influye de forma manifiesta en la circuitería necesaria ya que obliga a tener duplicado hardware porque hay casos en los cuales es indispensable utilizarlo en más de una ocasión en cada ciclo.

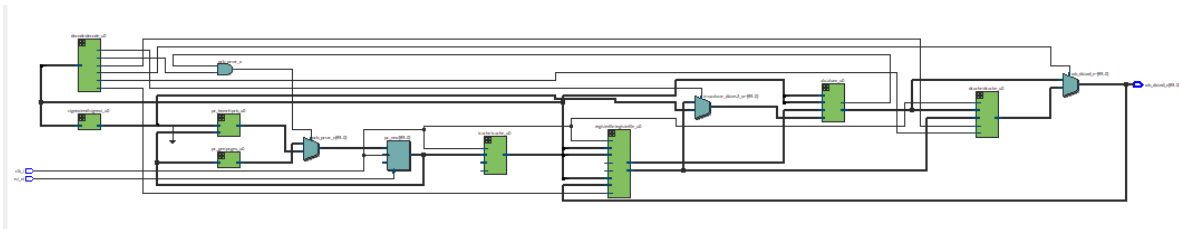
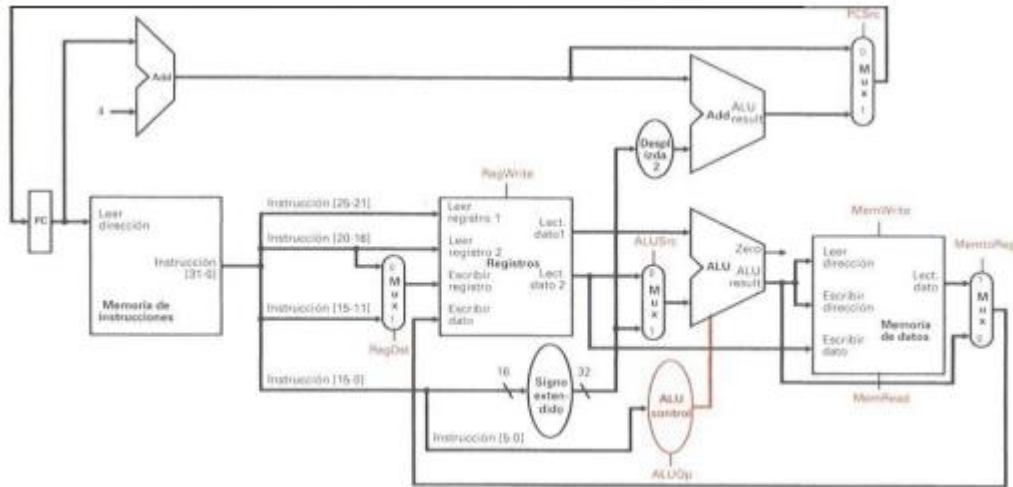
Este camino de datos se caracteriza, entre otras cosas, por tener una memoria de datos y otra de instrucciones, necesarias ambas porque al ejecutarse cada instrucción en un ciclo de reloj puede ser necesario acceder a ambas durante ese ciclo. Así en la memoria de instrucciones se accede a aquella instrucción almacenada en la posición asociada al valor del contador de programa (PC), y en la memoria de datos se pueden obtener los datos almacenados en los registros relacionados con la instrucción a ejecutar.

También es necesario el uso de tres Unidades Aritmético-Lógicas (ALUs) ya que dos de ellas son utilizadas para calcular el siguiente valor del PC y la otra se hace cargo de realizar las operaciones necesarias para las instrucciones aritmético-lógicas. Uno de los principales componentes de un camino de datos es la unidad de control, que a partir de los seis bits correspondientes al código de operación modifica los valores de las distintas señales de control del camino de datos. Hay algunos elementos como el banco de registros que necesitan una señal de escritura, o como la memoria de datos que requiere una señal de escritura y otra de lectura. Cada multiplexor necesita una línea de

control y, además, hay una unidad de control de la ALU que dependiendo de los valores de las líneas ALUOp controla el funcionamiento de esta. Estas señales de control se activan en función de las seis líneas de entrada a la unidad de control. Estas líneas se corresponden con los 6 bits más significativos de la codificación de cada instrucción

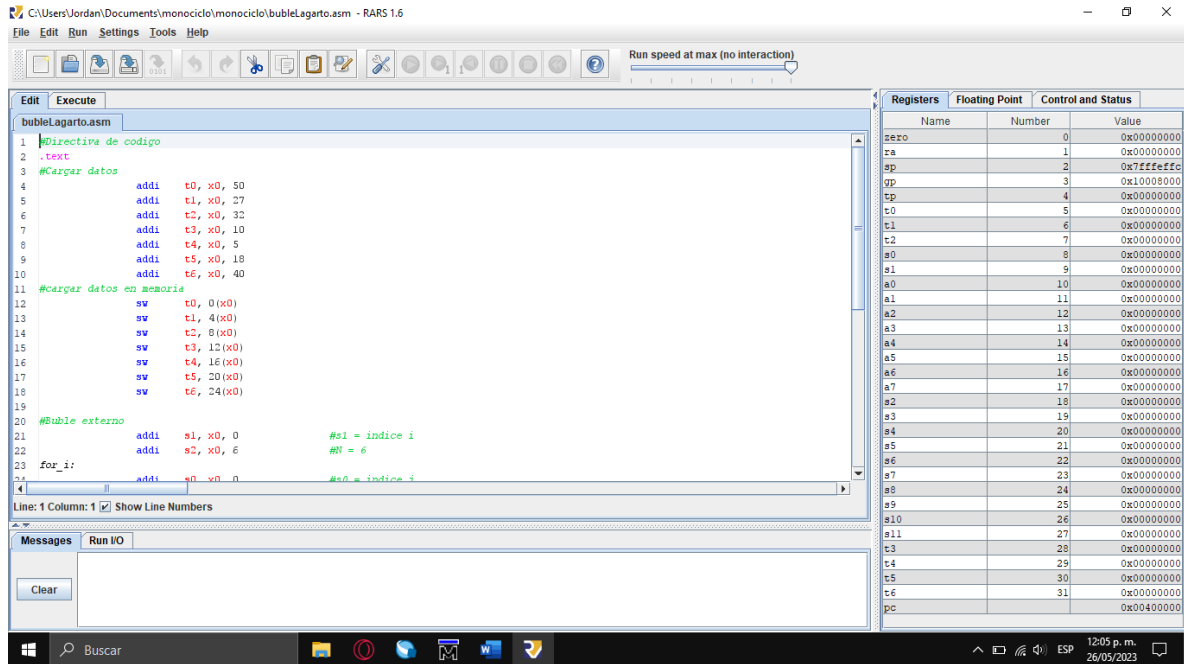
Desarrollo

En el desarrollo de un monociclo se tiene en cuenta las instrucciones del cache para el desarrollo de todas las instrucciones que necesita para ejecutar instrucciones en hexadecimal como se mostrara mas adelante.



Como se vio, tenemos un conjunto de componentes que conforman el monociclo, en los cuales se realiza un constante ciclo para realizar las instrucciones Por medio un ALU y un lector de archivos, que en este caso lee archivos de tipo hexadecimal, dentro del monociclo encontramos un decodificador para saber el tipo de instrucción que se está pidiendo y cuando llega al ALU este realiza la operación requerida.

Para generar el archivo usamos la herramienta RARS que nos permite pasar las instrucciones a código hexadecimal para que el monociclo pueda leerlo.



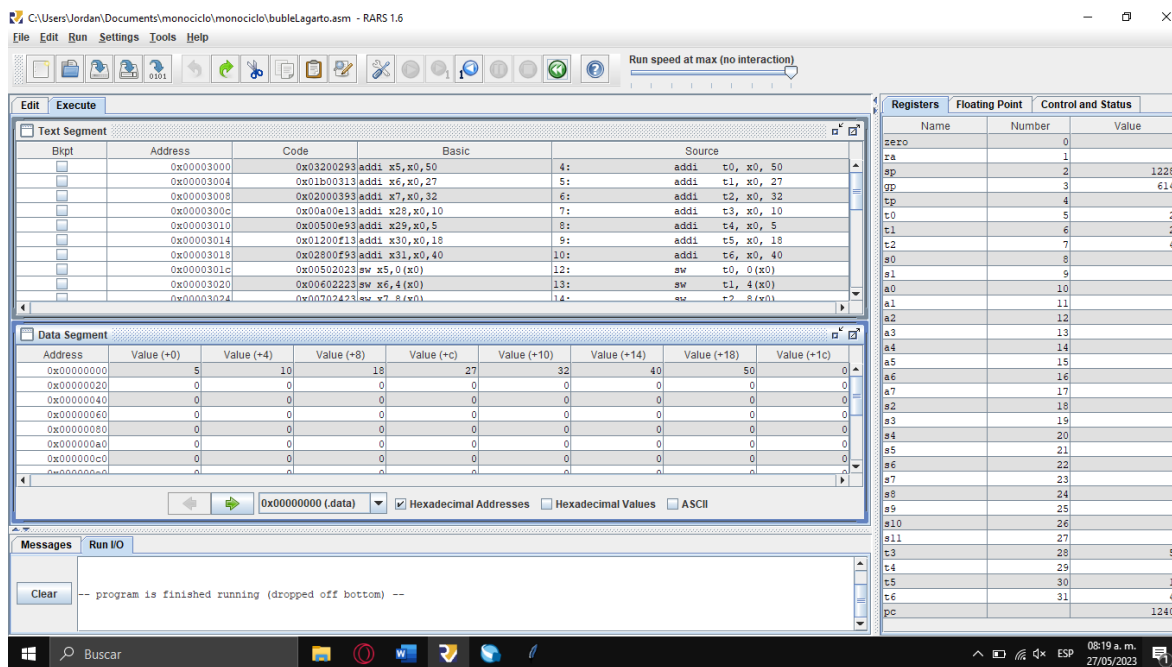
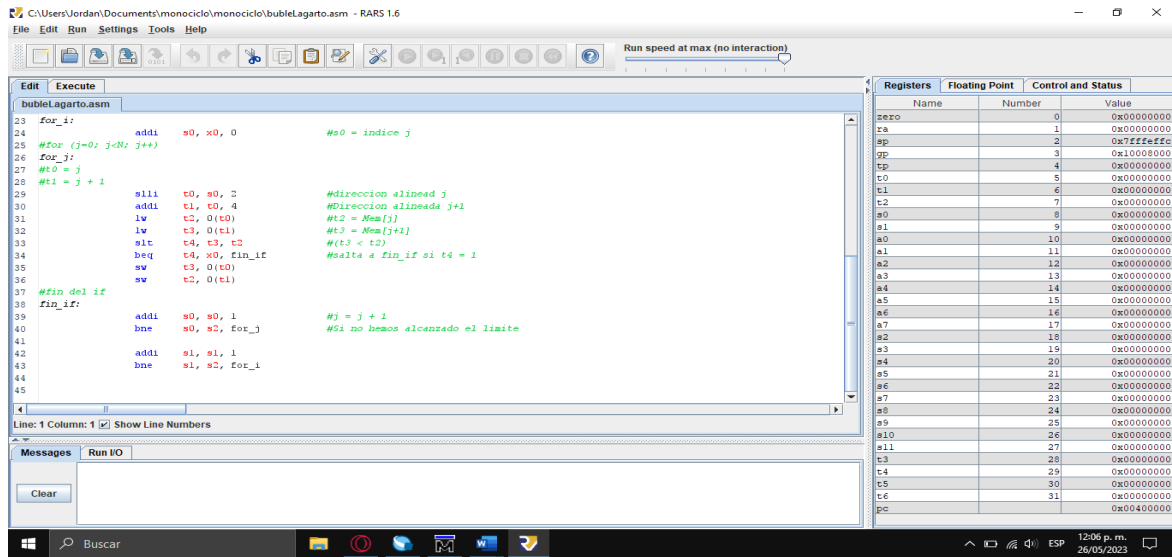
```
// b0010011: //Instrucciones Tipo I
begin
  case (aluop_i)
    4'b0000: //Suma - addi
      resultado_o = src1_i + src2_i;

    4'b0010: //Set if less than immediate - slti
      begin
        if (src1_i < src2_i)
          resultado_o = 64'h1;
        else
          resultado_o = 64'h0;
      end

    4'b0011: //Set if less than immediate unsigned - sltiu
      begin
        if (src1_i < src2_i)
          resultado_o = 64'h1;
        else
          resultado_o = 64'h0;
      end

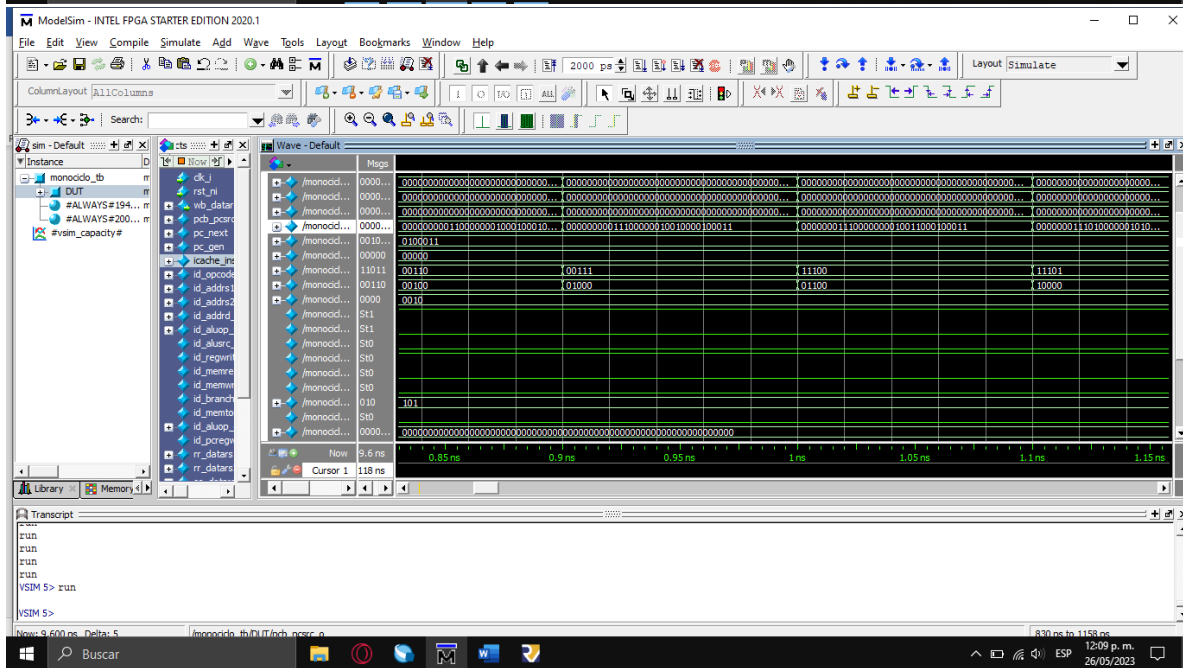
    4'b0100: //xori
      resultado_o = src1_i ^ src2_i;
    4'b0110: //ori
      resultado_o = src1_i | src2_i;
    4'b0111: //andi
      resultado_o = src1_i & src2_i;
    4'b0001: //slli
      resultado_o = src1_i << src2_i;
    4'b0101: //srli
      resultado_o = src1_i >> src2_i;
    4'b1101: //srai
      resultado_o = src1_i >>> src2_i;
  default:
    resultado_o = 0;
end
```

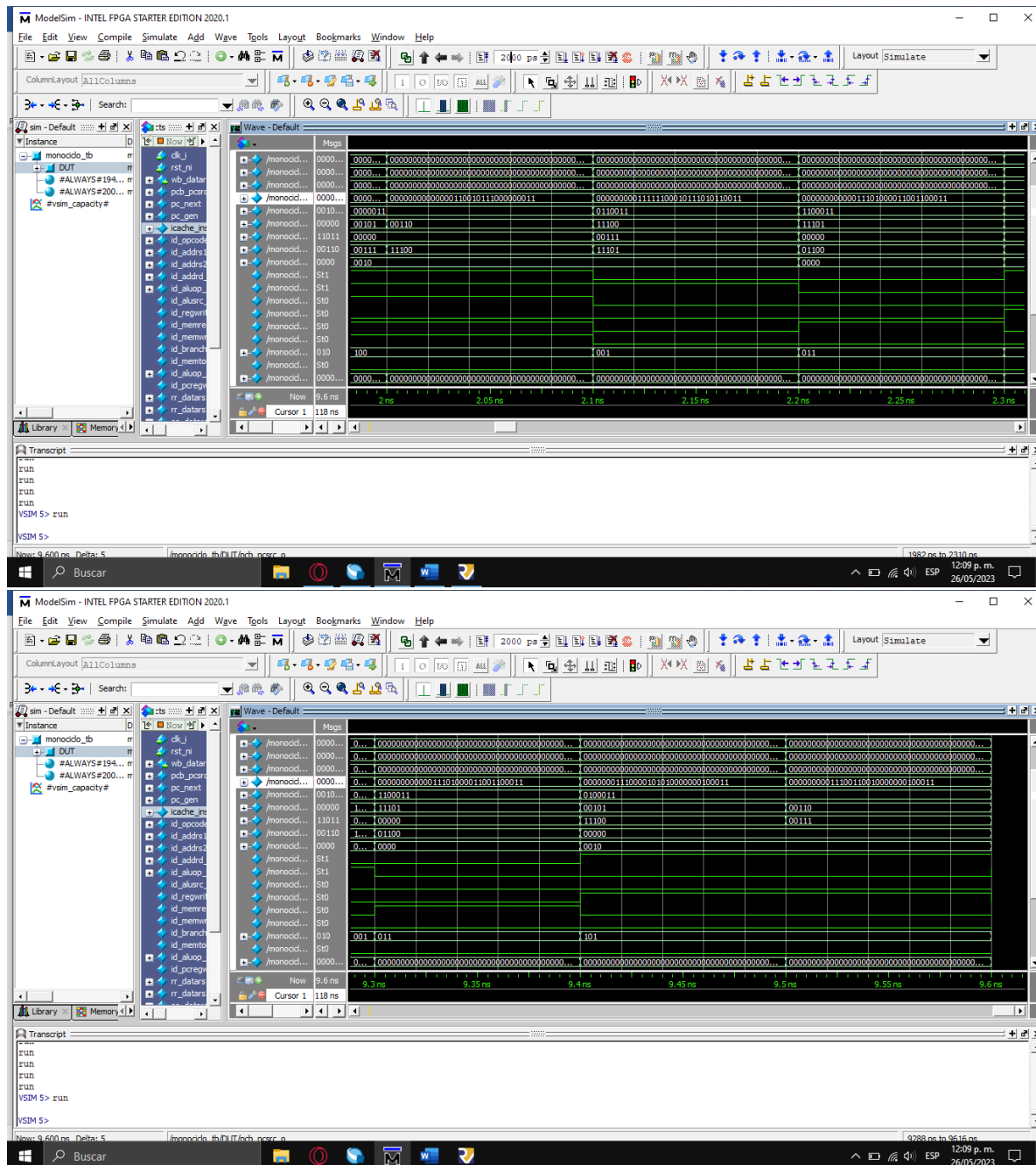
Las instrucciones son obtenidas de la ALU, aquí ya están en código a su el programa va a trasladar los números y comparar para acomodarlos en un orden descendente como su ejecución en Rars.



Y a continuación se muestra su ejecución en Modelsim.









Ahora se cambia los números del .asm como su ejecución en Rars.

C:\Users\Jordan\Documents\monociclo\monociclo\bubleLagarto.asm - RARS 1.6

File Edit Run Settings Tools Help

Run speed at max (no interaction)

Edit Execute

Secuencia.asm* bubleLagarto.asm

```
1 #Directiva de codigo
2 .text
3 #Cargar datos
4     addi    t0, x0, 56
5     addi    t1, x0, 67
6     addi    t2, x0, 22
7     addi    t3, x0, 18
8     addi    t4, x0, 24
9     addi    t5, x0, 199
10    addi    t6, x0, 48
11 #cargar datos en memoria
12    sw      t0, 0(x0)
13    sw      t1, 4(x0)
14    sw      t2, 8(x0)
15    sw      t3, 12(x0)
16    sw      t4, 16(x0)
17    sw      t5, 20(x0)
18    sw      t6, 24(x0)
19
20 #suble externo
21    addi    s1, x0, 0      #s1 = indice i
22    addi    s2, x0, 6      #N = 6
23 for_i:
24    addi    x0, x0, 0      #x0 = indice i
```

Line: 10 Column: 18 Show Line Numbers

Messages Run I/O

Clear

Registers	Floating Point	Control and Status
Name	Number	Value
zero	0	0
ra	1	0
sp	2	12284
gp	3	6144
tp	4	0
t0	5	20
t1	6	24
t2	7	199
a0	8	6
a1	9	0
a0	10	0
a1	11	0
a2	12	0
a3	13	0
a4	14	0
a5	15	0
a6	16	0
a7	17	0
a2	18	6
a3	19	0
a4	20	0
a5	21	0
a6	22	0
a7	23	0
a8	24	0
a9	25	0
a10	26	0
a11	27	0
t3	28	48
t4	29	1
t5	30	199
t6	31	48
pc		12392

08:14 a. m. 27/05/2023

C:\Users\Jordan\Documents\monociclo\monociclo\bubleLagarto.asm - RARS 1.6

File Edit Run Settings Tools Help

Run speed at max (no interaction)

Edit Execute

Text Segment

Addr	Address	Code	Basic	Source
	0x00003044	0x00241293	alll x5,x8,2	29: alll t0, a0, 2 ...
	0x00003048	0x00428313	addi x6,x5,4	30: addi t1, t0, 4 ...
	0x0000304c	0x0002a383	lw x7,0(x5)	31: lw t2, 0(t0) ...
	0x00003050	0x00032e03	lw x28,0(x6)	32: lw t3, 0(t1) ...
	0x00003054	0x007e2eb3	sllt x29,x28,x7	33: sllt t4, t3, t2 ...
	0x00003058	0x000e6e43	beq x29,x0,i2	34: beq t4, x0, fin_if ...
	0x0000305c	0x01c3a023	sw x28,0(x5)	35: sw t3, 0(t0) ...
	0x00003060	0x00732023	sw x7,0(x6)	36: sw t2, 0(t1) ...
	0x00003064	0x00140413	addi x8,x8,1	39: addi a0, a0, 1 ...
	0x00003068	0xf4741ae3	hne x8,x18,-36	40: hne a0, a2, for_i ...

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x00000000	18	22	24	48	56	67	199	0
0x00000020	0	0	0	0	0	0	0	0
0x00000040	0	0	0	0	0	0	0	0
0x00000060	0	0	0	0	0	0	0	0
0x00000080	0	0	0	0	0	0	0	0
0x000000a0	0	0	0	0	0	0	0	0
0x000000c0	0	0	0	0	0	0	0	0
0x000000e0	0	0	0	0	0	0	0	0

0x00000000 (.data) Hexadecimal Addresses Hexadecimal Values ASCII

Messages Run I/O

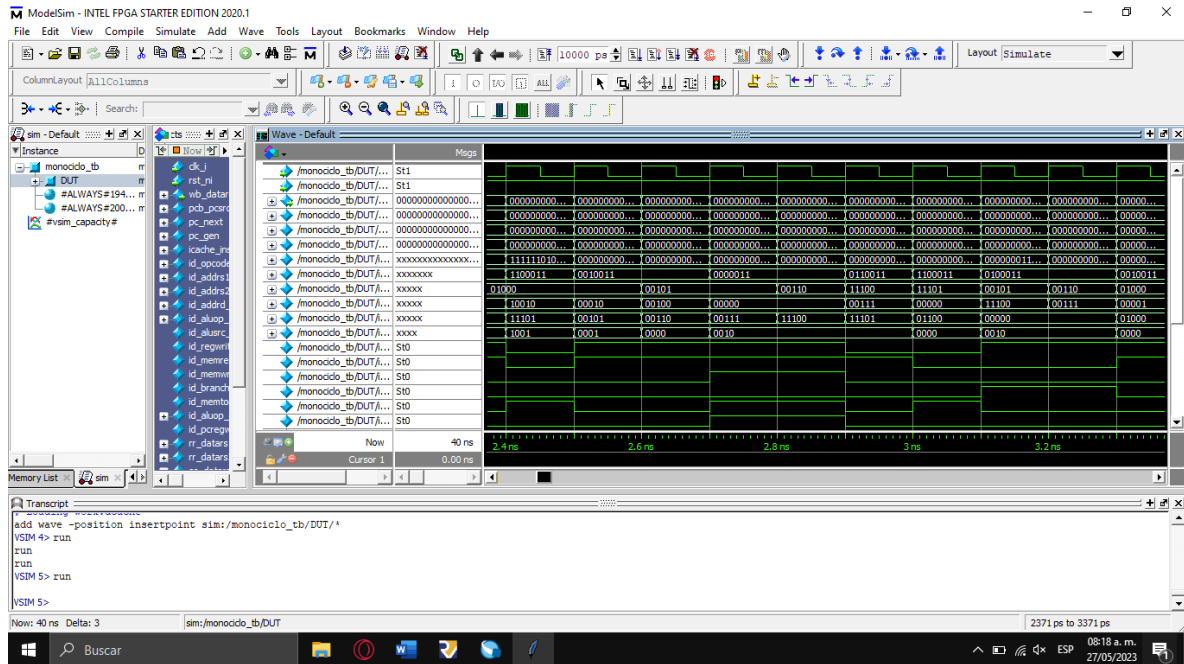
Clear

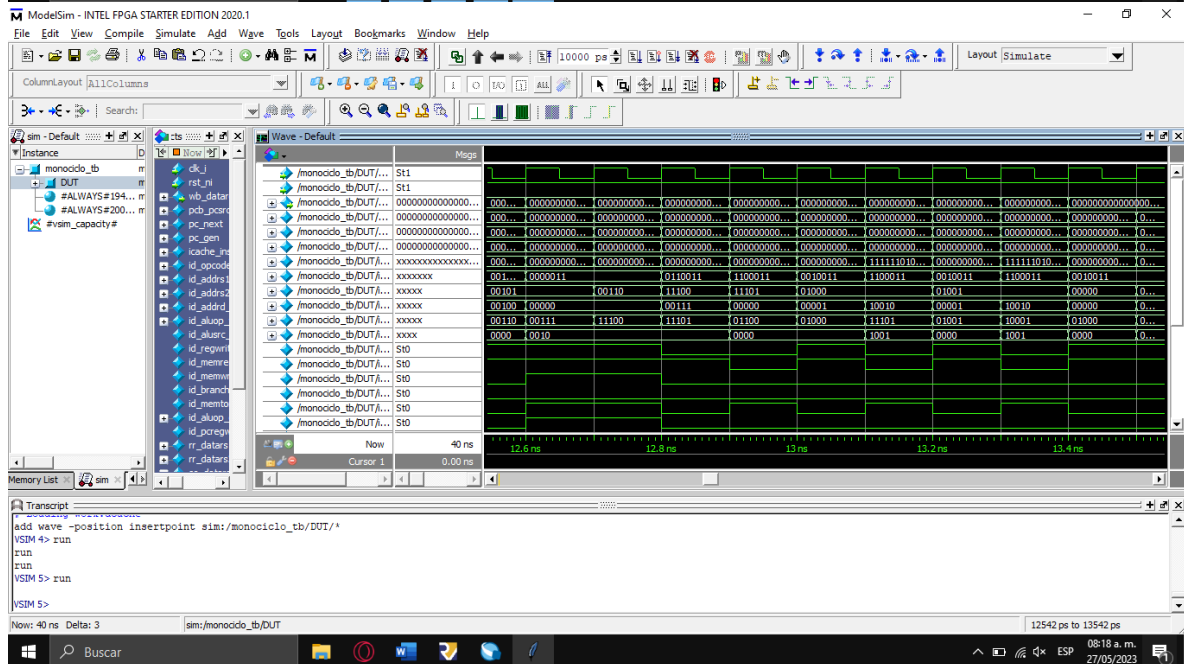
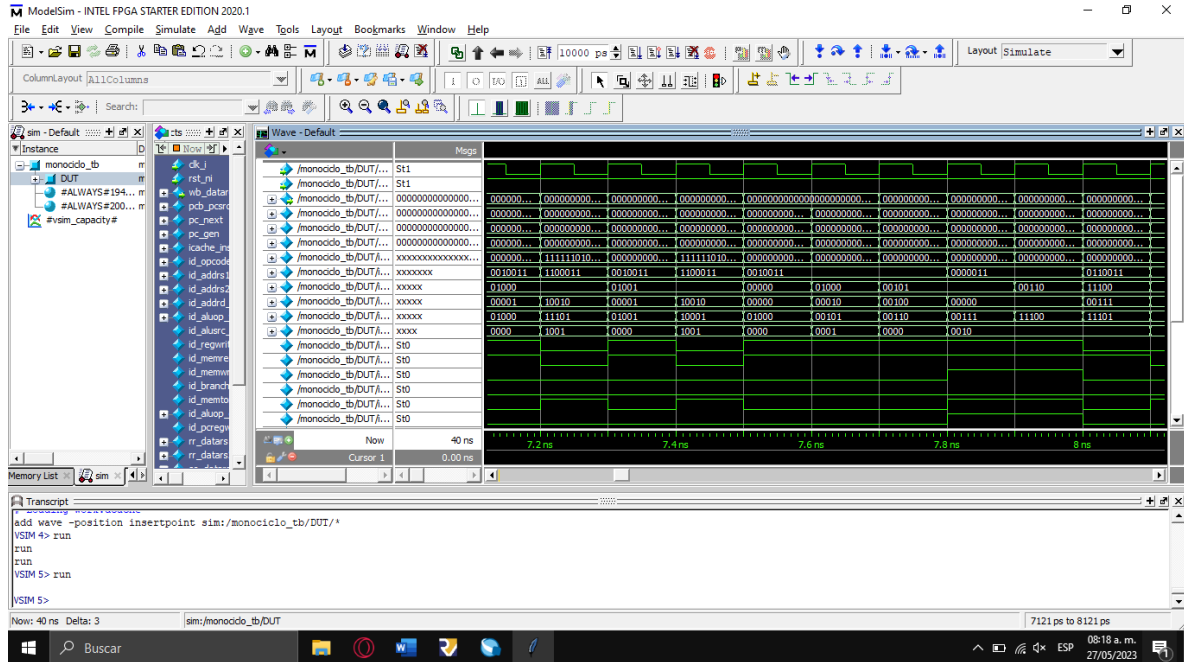
-- program is finished running (dropped off bottom) --

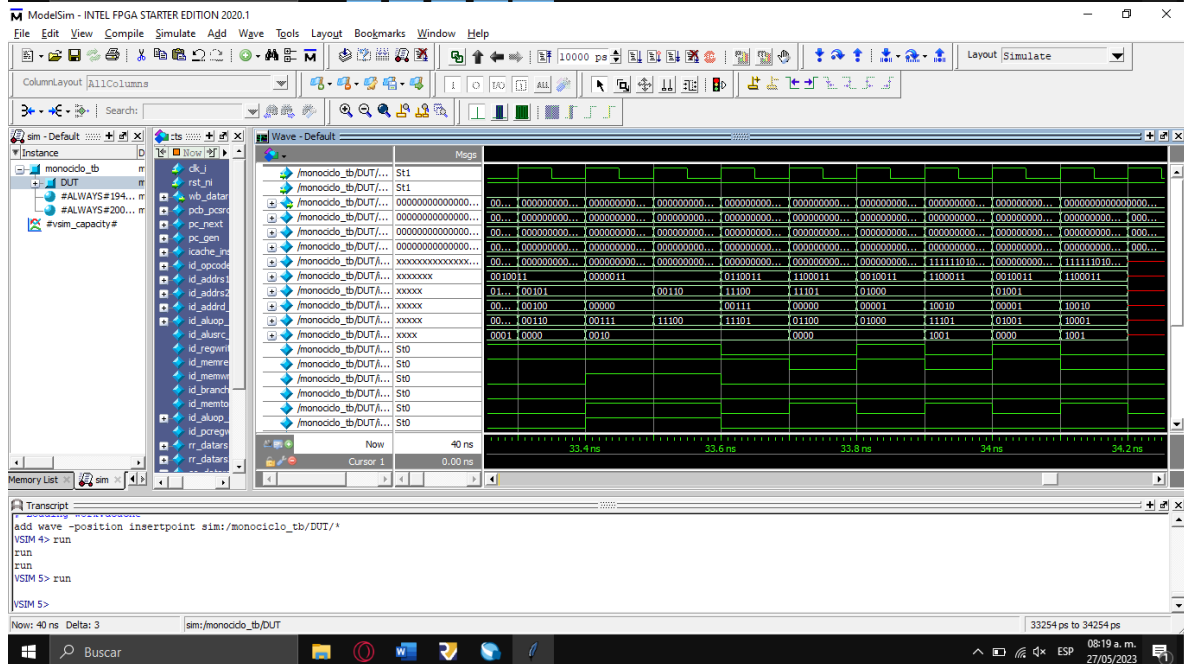
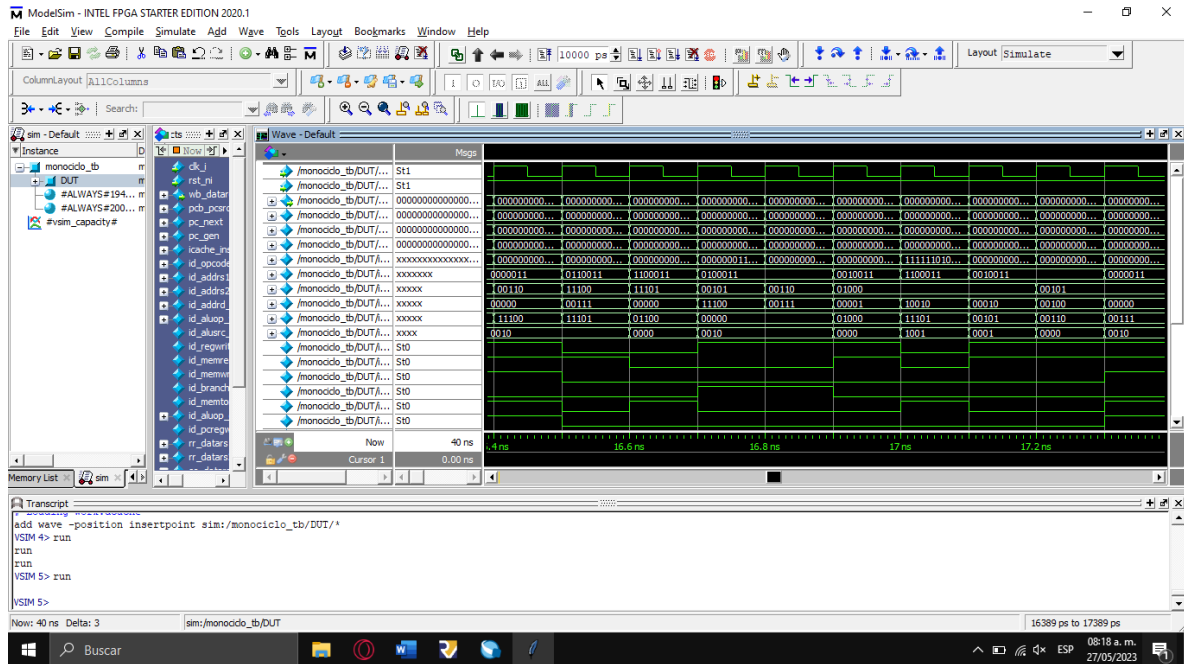
Registers	Floating Point	Control and Status
Name	Number	Value
zero	0	0
ra	1	0
sp	2	12284
gp	3	6144
tp	4	0
t0	5	20
t1	6	24
t2	7	67
a0	8	6
a1	9	6
a0	10	0
a1	11	0
a2	12	0
a3	13	0
a4	14	0
a5	15	0
a6	16	0
a7	17	0
a2	18	6
a3	19	0
a4	20	0
a5	21	0
a6	22	0
a7	23	0
a8	24	0
a9	25	0
a10	26	0
a11	27	0
t3	28	199
t4	29	0
t5	30	199
t6	31	48
pc		12408

08:16 a. m. 27/05/2023

Y ahora se muestra su ejecución en Modelsim.







Por último, se creó el siguiente programa de ejemplo en el Rars y se muestra su ejecución

C:\Users\Jordan\Documents\monociclo\monociclo\Secuencia.asm - RARS 1.6

File Edit Run Settings Tools Help

Run speed at max (no interaction)

Registers Floating Point Control and Status

Name	Number	Value
zero	0	0
ra	1	0
sp	2	2147479548
gp	3	268468224
tp	4	0
t0	5	0
t1	6	0
t2	7	0
a0	8	0
a1	9	0
a2	10	0
a3	11	0
a4	12	0
a5	13	0
a6	14	0
a7	15	0
a8	16	0
a9	17	0
a10	18	0
a11	19	0
a12	20	0
a13	21	0
a14	22	0
a15	23	0
a16	24	0
a17	25	0
a18	26	0
a19	27	0
a20	28	0
a21	29	0
a22	30	0
a23	31	0
pc		4194304

```

1 # Directiva de código
2 .text
3
4 # Cargar datos
5 addi t0, x0, 50
6 addi t1, x0, 27
7
8 # Sumar los valores
9 add t2, t0, t1
10
11 # Restar los valores
12 sub t3, t0, t1
13
14 # Multiplicar los valores
15 mul t4, t0, t1
16
17 # Dividir los valores
18 div t5, t0, t1
19
20 # Mostrar el resultado por pantalla
21 li a7, 4 # Código del sistema para imprimir cadena
22 la a0, resultado_msg # Dirección de la cadena resultado_msg
23 ecall

```

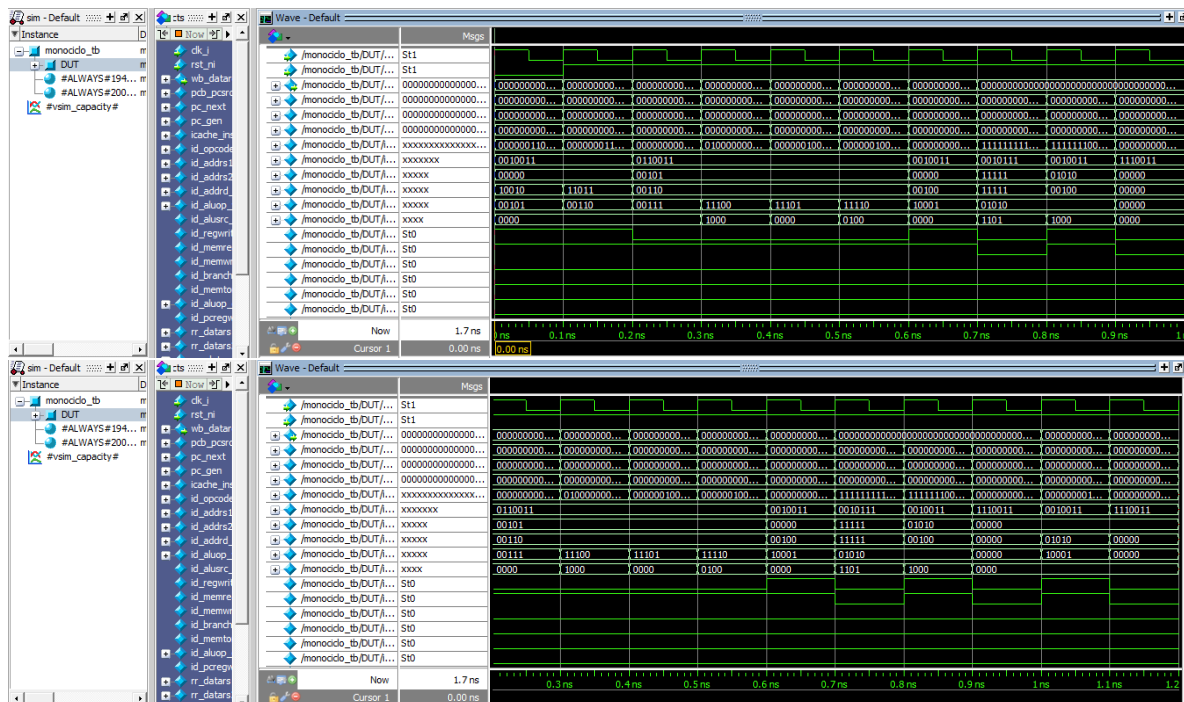
Line: 19 Column: 1 Show Line Numbers

Messages Run I/O

Clear

Buscar

ESP 08:02 a. m. 27/05/2023





Referencias

Abril Torres, P. (n.d.). UNIVERSIDAD DE CASTILLA-LA MANCHA ESCUELA SUPERIOR DE INGENIERÍA INFORMÁTICA GRADO EN INGENIERÍA INFORMÁTICA: TECNOLOGÍA ESPECÍFICA DE INGENIERÍA DE COMPUTADORES TRABAJO FIN DE GRADO.
<https://www.dsi.uclm.es/personal/FcoAlfaro/pfc/TFG%20Pablo%20Abril.pdf>

Manuel, J., & Cuadros, M. (n.d.). Diseño monociclo del procesador. Retrieved May 27, 2023, from <http://www.fdi.ucm.es/profesor/mendias/FC2/FC2tema5-imprimible.pdf>